Technical Documentation

RAG-Based Customer Support Assistant

Version: 1.0
Date: April 2026
Author: Moin Uddin

RAG-Based Customer Support Assistant
with LangGraph & Human-in-the-Loop Escalation

Table of Contents

1. Introduction
2. System Architecture Explanation
3. Design Decisions
4. Workflow Explanation
5. Conditional Logic
6. HITL Implementation
7. Challenges & Trade-offs
8. Testing Strategy
9. Future Enhancements

---

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines the strengths of two systems:

1. **Retrieval System:** A search engine that finds relevant information from a knowledge base (documents, FAQs, manuals).
2. **Generation System:** A Large Language Model (LLM) that generates natural language responses.

The key insight behind RAG is: **LLMs are excellent at generating human-like text, but they can hallucinate facts.** By first *retrieving* verified information from a trusted knowledge base and then *augmenting* the LLM's prompt with that information, we get responses that are both fluent and factually grounded.

RAG Pipeline:

```
User Query
    ↓
[Embed Query → Search Vector DB → Retrieve Top-K Documents]
    ↓
[Inject Documents into LLM Prompt as Context]
    ↓
[LLM Generates Response Based on Context]
    ↓
Factually Grounded Response
```

Without RAG: The LLM answers from its training data (which may be outdated or wrong).

With RAG: The LLM answers from your specific documents (accurate, up-to-date, verifiable).

1.2 Why is RAG Needed?

Problem	How RAG Solves It
LLMs hallucinate facts	Responses are grounded in retrieved documents
Training data becomes stale	Knowledge base can be updated without retraining
LLMs lack domain-specific knowledge	Custom PDFs provide domain expertise
No source attribution	Retrieved chunks provide traceable sources
Expensive fine-tuning	RAG is cheaper than fine-tuning for most use cases

1.3 Use Case Overview

Our system implements a **Customer Support Assistant** for "TechFlow Solutions," a fictional SaaS company. The assistant:

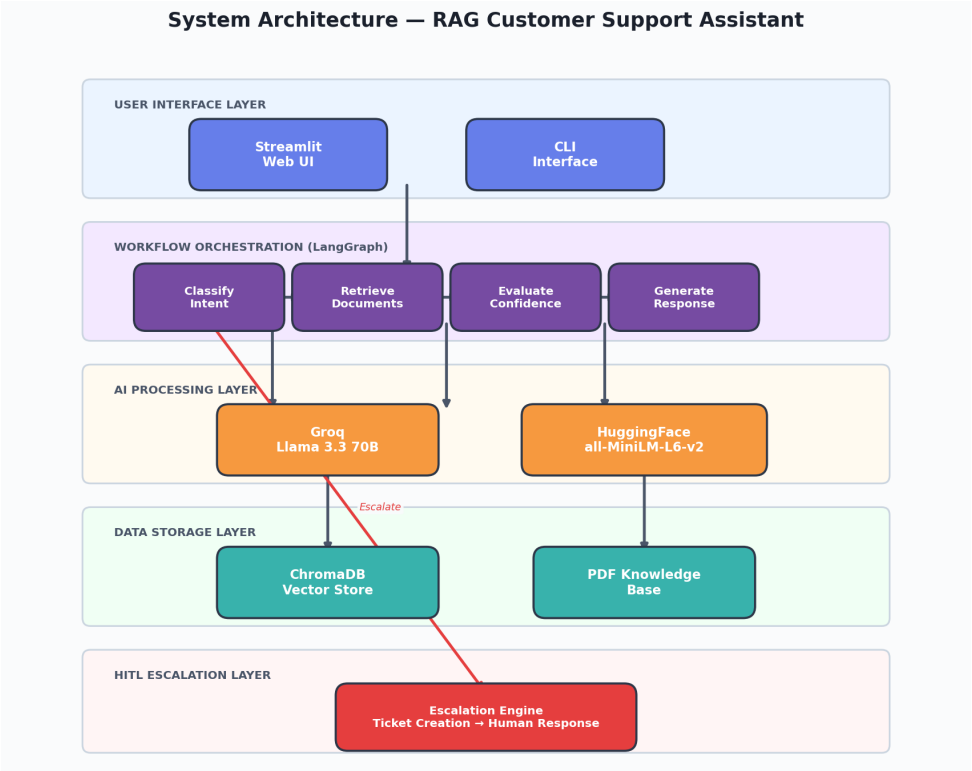
- **Answers FAQs** about products, pricing, account management, and policies
- **Provides troubleshooting steps** for common technical issues
- **Routes complaints** and complex queries to human agents
- **Shows transparency** by displaying the source documents used for each answer
- **Escalates intelligently** when it detects uncertainty or sensitive topics

Example interactions:

- *"How do I reset my password?"* → RAG retrieves password reset docs → Generates step-by-step answer
- *"I'm very frustrated with your service!"* → Intent classified as COMPLAINT → Escalated to human agent
- *"What is quantum physics?"* → OUT_OF_SCOPE → Escalated (outside knowledge base)

---

2. System Architecture Explanation



2.1 Detailed Architecture

The system consists of **five interconnected layers:**

Layer 1: User Interface

The Streamlit web application provides:

- A chat interface using `st.chat_message` and `st.chat_input` for natural conversation
- A sidebar for PDF upload, knowledge base management, and system status
- An escalation panel that appears when HITL is triggered
- Source document display via expandable sections

The UI maintains conversation history via `st.session_state`, allowing multi-turn conversations within a session.

Layer 2: Workflow Orchestration (LangGraph)

This is the brain of the system. The LangGraph `StateGraph` manages the entire query lifecycle through 5 nodes and 2 conditional edges. It ensures that every query follows the correct path: classification → retrieval → evaluation → generation OR escalation.

The graph operates on a shared `GraphState` TypedDict, where each node reads from and writes to specific fields. This state-based architecture ensures:

- Complete traceability (every decision is logged)
- Idempotent execution (can be restarted safely)
- Clean separation of concerns (each node has a single responsibility)

Layer 3: AI Processing

Two AI models work together:

- **Groq Llama 3.3 70B (LLM):** Handles intent classification and response generation via Groq's ultra-fast LPU inference. Used at temperature=0.3 for consistent, factual outputs.
- **HuggingFace Embeddings (local):** Converts text into 384-dimensional vectors using the `all-MiniLM-L6-v2` model running locally on the machine. No API calls needed. Both queries and document chunks use the same model, ensuring compatible vector spaces.

Layer 4: Data Storage

ChromaDB serves as the vector database:

- Stores document chunk embeddings alongside raw text and metadata
- Performs cosine similarity search to find relevant chunks
- Persists data to disk (survives application restarts)
- Supports metadata filtering (e.g., by source file or page)

Layer 5: HITL System

The escalation engine monitors multiple signals:

- Intent classification results
- Retrieval confidence scores
- Explicit user requests

When escalation is triggered, it creates a context-rich ticket and presents a human response interface.

2.2 Component Interactions

```
User → [Streamlit] → [LangGraph]
                          ■
                     ■■■■■■■■■■■
                     ■    ■    ■
               [Gemini] [ChromaDB] [HITL]
                     ■    ■    ■
                     ■■■■■■■■■■■
                          ■
                     [Streamlit] → User
```

Key interaction patterns:

1. **Streamlit → LangGraph:** The UI calls `run_graph(query, chat_history)` and receives the final state
2. **LangGraph → Gemini:** Nodes call the LLM for classification and generation
3. **LangGraph → ChromaDB:** The retrieval node queries the vector store
4. **LangGraph → HITL:** The escalation node creates tickets and messages
5. **Streamlit → HITL:** The UI presents the human response form and calls `integrate_human_response()`

---

3. Design Decisions

3.1 Chunk Size Choice: 500 Characters

Decision: 500 characters per chunk with 100-character overlap (20%).

Reasoning:

- **Too small (100-200 chars):** Chunks lose context. A question about password reset might match a chunk that only says "passwords" without the actual reset steps.
- **Too large (1000+ chars):** Chunks become noisy. A large chunk about account management might match both "create account" and "delete account" queries, diluting relevance.
- **500 chars (sweet spot):** Typically captures a complete FAQ answer or troubleshooting step. Large enough for context, small enough for precision.

Why 20% overlap:

```
Without overlap:
  Chunk 1: "...follow these steps to reset your |"
  Chunk 2: "| password: 1) Go to the reset page..."
  → The sentence is split! Neither chunk is fully useful.

With 100-char overlap:
  Chunk 1: "...follow these steps to reset your password: 1) Go to"
  Chunk 2: "reset your password: 1) Go to the reset page. 2) Enter..."
  → Both chunks contain the critical sentence.
```

3.2 Embedding Strategy

Model: HuggingFace `all-MiniLM-L6-v2` (local)

Why a local embedding model?

- Runs entirely on your machine — zero API cost, no rate limits
- No external API dependency — works offline, no network latency
- Produces 384-dimensional vectors (compact yet effective for semantic search)
- ~80MB model cached locally after first download

Why not OpenAI or Google cloud embeddings?

- Cloud-based models incur API costs and rate limits during high-volume ingestion
- Network latency adds overhead to every query
- Local models provide consistent performance regardless of internet availability

Consistency principle: The same embedding model must be used for both document ingestion and query encoding. Mixing models (e.g., embedding documents with one model but querying with another) would produce incompatible vector spaces.

3.3 Retrieval Approach

Method: Cosine similarity with top-K=3

Why cosine similarity?

- Standard for text embeddings
- Scale-invariant (works regardless of vector magnitude)
- Built into ChromaDB as the default metric

Why K=3?

- K=1: Too risky — might miss the best answer if the top result is slightly off-topic
- K=3: Provides redundancy — if one chunk is partial, the others fill gaps
- K=5+: Diminishing returns — more chunks → more noise in the LLM prompt → slower responses

3.4 Prompt Design Logic

System Prompt Strategy:

The system prompt establishes the assistant's identity and ground rules before any query is processed:

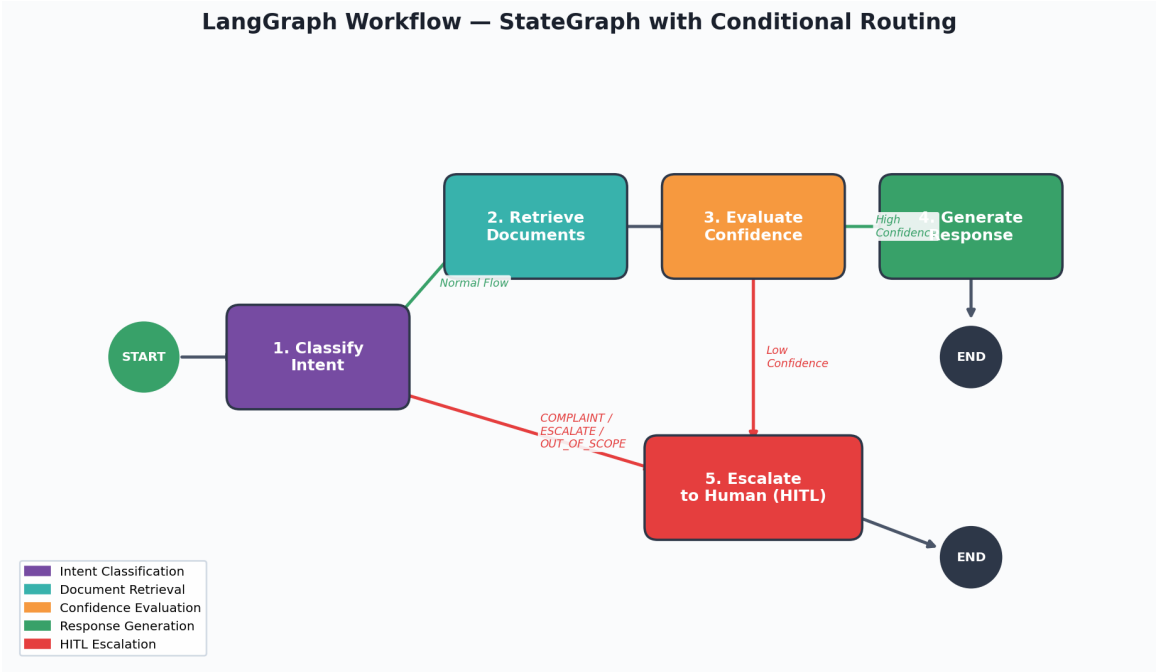
```
1. IDENTITY: "You are a helpful and professional AI Customer Support Assistant."
   → Sets the tone and behavior expectations.

2. GROUNDING RULE: "Answer ONLY based on the provided context."
   → Prevents hallucination by constraining the LLM.

3. UNCERTAINTY HANDLING: "If the context does not contain enough information..."
   → Ensures the LLM doesn't make up answers when context is insufficient.

4. TONE: "Be polite, concise, and professional."
   → Customer support requires empathy and clarity.

5. FORMATTING: "Format your response with bullet points or numbered steps."
   → Structured responses are easier for customers to follow.
```

Why temperature=0.3?

- Temperature controls randomness in LLM outputs
- 0.0 = deterministic (same input always same output)
- 0.3 = slightly creative (natural language variety without wild deviations)
- 1.0+ = very creative (unsuitable for factual support responses)

---

4. Workflow Explanation



4.1 LangGraph Usage

LangGraph is a library for building stateful, graph-based AI workflows. Unlike simple sequential chains (A → B → C), LangGraph supports:

- **Conditional branching:** Different paths based on runtime decisions
- **Cycles:** Loops for retry logic (not used in our system, but available)
- **Shared state:** All nodes read/write to the same state dictionary
- **Built-in HITL:** `interrupt()` function for pausing execution

Why not a simple chain?

A sequential chain (classify → retrieve → generate) would process every query the same way. But customer support requires routing:

- Complaints should skip retrieval and go straight to a human
- Low-confidence retrievals should escalate instead of generating bad answers
- Out-of-scope queries should be handled differently than FAQ queries

LangGraph's conditional edges make this routing natural and explicit.

4.2 Node Responsibilities

Node	Responsibility	LLM Call?	DB Access?
`classify_intent`	Determine what the user wants	Yes (classification)	No
`retrieve_documents`	Find relevant knowledge base chunks	No	Yes (ChromaDB)

`evaluate_confidence`	Score retrieval quality	No	No
`generate_response`	Create the final answer	Yes (generation)	No
`escalate_to_human`	Create ticket + escalation message	No	No

Key observations:

- Only 2 nodes require LLM calls (expensive operations are minimized)
- Only 1 node accesses the database
- 2 nodes are pure logic (confidence evaluation + escalation) — fast and testable

4.3 State Transitions

Normal flow (FAQ query):

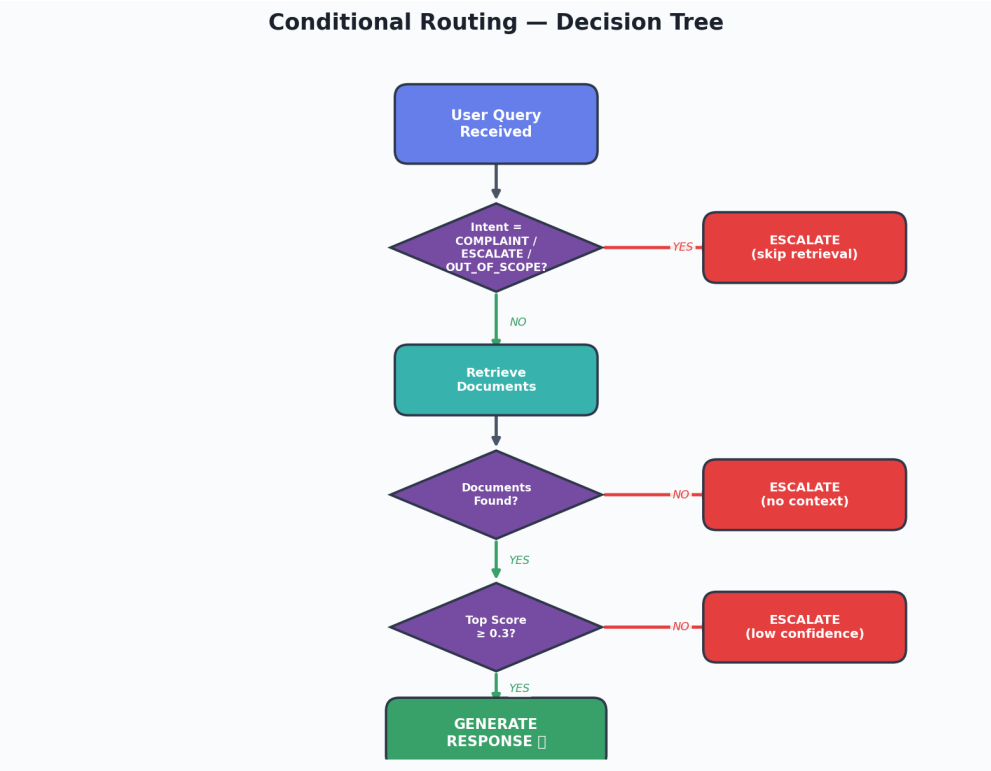
```
Initial → {query: "How to reset password?", intent: "", response: ""}
After classify_intent → {intent: "GENERAL_QUERY", needs_escalation: false}
After retrieve_documents → {retrieved_docs: [...], context: "...", sources: [...]}
After evaluate_confidence → {retrieval_confidence: 0.89, needs_escalation: false}
After generate_response → {response: "To reset your password, follow these steps..."}
```

Escalation flow (complaint):

```
Initial → {query: "I'm furious! Your service is terrible!", intent: "", response: ""}
After classify_intent → {intent: "COMPLAINT", needs_escalation: true}
→ SKIP retrieve_documents and evaluate_confidence
After escalate_to_human → {response: "I'm sorry to hear... escalating to human agent.",
                           escalation_ticket: {ticket_id: "ESC-4291", ...}}
```

---

5. Conditional Logic



5.1 Intent Detection

The LLM classifies each query into exactly one of six categories:

Intent	Examples	Action
`GENERAL_QUERY`	"What are your plans?", "How to create account?"	Retrieve → Generate
`TECHNICAL_ISSUE`	"My login isn't working", "Video call is lagging"	Retrieve → Generate
`BILLING`	"How do I get a refund?", "What's my invoice?"	Retrieve → Generate
`COMPLAINT`	"I'm very frustrated!", "Your service is awful"	ESCALATE immediately
`ESCALATE`	"Let me talk to a human", "I need a manager"	ESCALATE immediately
`OUT_OF_SCOPE`	"What's the weather?", "Explain quantum physics"	ESCALATE immediately

Fallback mechanism: If the LLM returns an unrecognized category, keyword-based fuzzy matching attempts to map it to the closest valid category. If all else fails, it defaults to `GENERAL_QUERY`.

5.2 Routing Decisions

Decision Point 1: After Intent Classification

```
def route_after_intent(state):
    if state["needs_escalation"]:  # True for COMPLAINT/ESCALATE/OUT_OF_SCOPE
        return "escalate_to_human"
    else:
        return "retrieve_documents"
```

Decision Point 2: After Confidence Evaluation

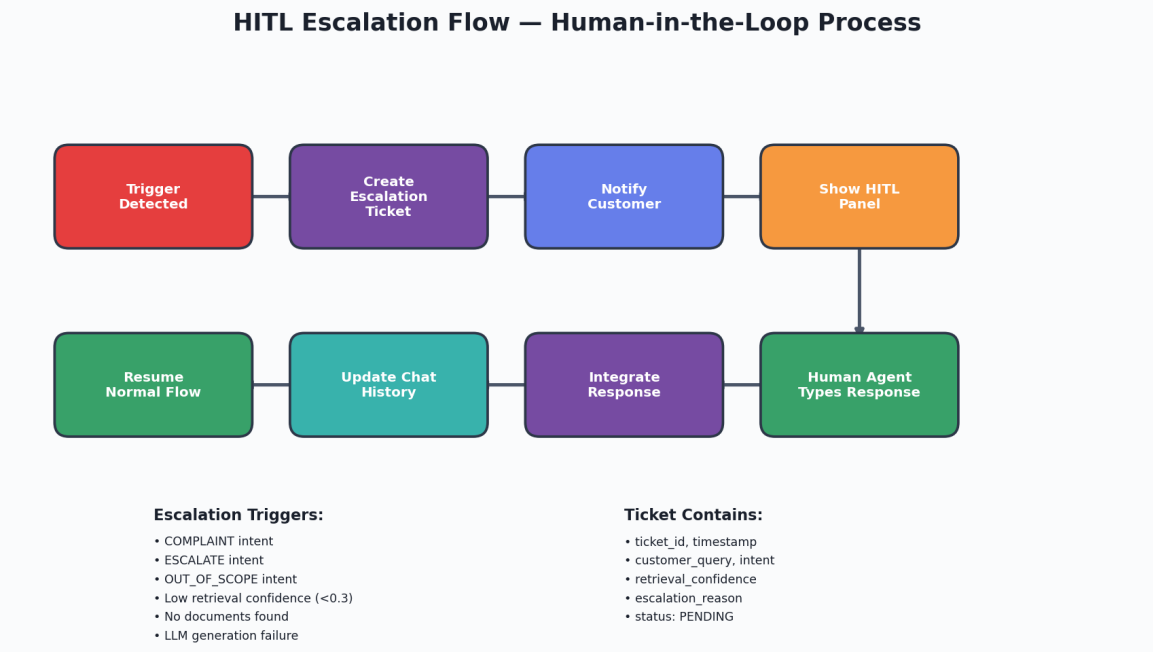
```
def route_after_confidence(state):
    if state["needs_escalation"]:  # True when confidence < 0.4 or no docs
        return "escalate_to_human"
    else:
        return "generate_response"
```

Why two decision points?

1. **First checkpoint (intent):** Catches obviously non-retrievable queries early, saving unnecessary vector search and LLM costs.
2. **Second checkpoint (confidence):** Catches cases where the query seems answerable but the knowledge base doesn't have good matches.

---

6. HITL Implementation



6.1 Role of Human Intervention

The HITL system serves as a **safety net** for the AI. It recognizes that AI cannot (and should not) handle every customer interaction autonomously. Human intervention is essential for:

1. **Emotional situations:** Complaints require empathy that AI may not reliably deliver
2. **Edge cases:** Queries that fall outside the training data or knowledge base
3. **High-stakes decisions:** Issues involving billing disputes, account changes, or legal matters
4. **Quality assurance:** Catches potential hallucinations or inaccurate responses

6.2 Benefits of HITL

Benefit	Explanation
Customer satisfaction	Frustrated customers get human attention
Risk mitigation	AI doesn't provide wrong answers to sensitive queries
Continuous improvement	Human responses can be fed back to improve the knowledge base
Trust building	Transparency about AI limitations increases user trust
Legal compliance	Some regulations require human oversight for certain decisions

6.3 Limitations of HITL

Limitation	Impact
Latency	Human response takes minutes/hours vs. AI's seconds
Availability	Humans aren't 24/7 (unless specifically staffed)
Cost	Human agents are more expensive than AI processing
Consistency	Different human agents may give different answers
Scalability	Human capacity is limited; can't handle sudden query spikes

6.4 Implementation Details

Escalation Triggers (Multi-Factor):

```
def should_escalate(state):
    return any([
        state["intent"] in ["COMPLAINT", "ESCALATE", "OUT_OF_SCOPE"],
        state["retrieval_confidence"] < 0.4,
        state.get("needs_escalation", False),
        len(state.get("retrieved_docs", [])) == 0,
    ])
```

Ticket Structure:

```
{
    "ticket_id": "ESC-4291",
    "timestamp": "2026-04-22 10:15:30",
    "customer_query": "I want to cancel and I'm very frustrated!",
    "detected_intent": "COMPLAINT",
    "retrieval_confidence": 0.0,  # N/A (skipped)
    "escalation_reason": "Customer has expressed dissatisfaction...",
    "retrieved_context": "Summary of any retrieved docs",
    "status": "PENDING_HUMAN_REVIEW",
```

```
}
```

UI Integration:

When escalation occurs, the Streamlit app displays:

1. The AI's escalation message to the user
2. A highlighted escalation panel with context
3. A text area for the human agent to compose a response
4. Send/Skip buttons

---

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs. Speed

Approach	Accuracy	Speed	Our Choice
Exhaustive search	Highest	Slowest	✗
Approximate NN (HNSW)	High	Fast	✓ (ChromaDB default)
Keyword search (BM25)	Medium	Fastest	✗
Hybrid (vector + keyword)	Highest	Moderate	Future enhancement

Trade-off: We chose approximate nearest neighbors (via ChromaDB's HNSW index) for the best balance. It's ~95% as accurate as exhaustive search but orders of magnitude faster for large collections.

7.2 Chunk Size vs. Context Quality

Chunk Size	Precision	Context	Embedding Quality
100 chars	High	Low (fragments)	Poor (too short)
300 chars	High	Medium	Good
500 chars	Medium-High	Good	Good
1000 chars	Medium	High	Good but noisy
Full page	Low	Very High	Noisy

Trade-off: 500 characters gives us enough context to answer most questions while keeping embeddings focused enough for precise retrieval. Smaller chunks would improve precision for very specific questions but would lose the surrounding context needed for complete answers.

7.3 Cost vs. Performance

Component	Free Tier	Paid Option	Performance Difference
LLM	Groq Llama 3.3 70B (free tier)	GPT-4o ($)	GPT-4o is better for nuanced reasoning
Embeddings	HuggingFace local (free)	OpenAI text-embedding-3-large ($)	Marginal improvement
Vector DB	ChromaDB (local, free)	Pinecone ($)	Pinecone better for scale/uptime
Hosting	Local (free)	Cloud VM ($)	Required for production

Trade-off: The entire system runs on free tiers, making it ideal for development and demonstration. For production use, the modular design allows upgrading individual components without changing the overall architecture.

---

8. Testing Strategy

8.1 Testing Approach

The testing strategy follows a **pyramid approach:**

```
        /■■■■■■■■■■\
       / End-to-End  \     ← Few: Full workflow tests
      /   (Manual)     \
     /■■■■■■■■■■■■■■■■■■■\
    / Integration Tests    \  ← Some: Component interaction tests
   /■■■■■■■■■■■■■■■■■■■■■■■■■■\
  /     Unit Tests            \  ← Many: Individual function tests
 /■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■\
```

Unit Tests (27 tests, all passing):

- `test_document_processor.py` (8 tests): PDF loading, chunking, metadata
- `test_retriever.py` (6 tests): Confidence scoring, context formatting
- `test_intent_classifier.py` (8 tests): Fuzzy matching, escalation reasons
- `test_graph_workflow.py` (5 tests): Routing logic, state schema

8.2 Sample Queries for Testing

Query	Expected Intent	Expected Behavior
"How do I reset my password?"	GENERAL_QUERY	RAG retrieval → confident answer

"What subscription plans do you offer?"	GENERAL_QUERY	RAG retrieval → plan details
"My video calls keep dropping"	TECHNICAL_ISSUE	RAG retrieval → troubleshooting steps
"I want a refund for last month"	BILLING	RAG retrieval → refund policy
"Your service is absolutely terrible!"	COMPLAINT	Immediate escalation to HITL
"Let me speak to a manager"	ESCALATE	Immediate escalation to HITL
"What is the capital of France?"	OUT_OF_SCOPE	Escalation (outside knowledge base)
"How do I export my data?"	GENERAL_QUERY	RAG retrieval → export instructions

8.3 Test Results

```
============================= test session starts ==============================
collected 27 items

tests/test_document_processor.py  ........     [29%]
tests/test_graph_workflow.py      .....        [48%]
tests/test_intent_classifier.py   ........     [77%]
tests/test_retriever.py           ......       [100%]

============================== 27 passed in 2.72s ==============================
```

---

9. Future Enhancements

9.1 Multi-Document Support

- **Current:** Single PDF knowledge base
- **Future:** Support multiple PDFs with per-document metadata filtering
- **Implementation:** Add a `source_collection` field to metadata; allow users to query specific document sets

9.2 Feedback Loop

- **Current:** No learning from interactions
- **Future:** Track user satisfaction (thumbs up/down on responses)
- **Implementation:**

1. Add feedback buttons after each response
2. Store (query, response, feedback) triplets
3. Use positive examples to create few-shot prompts

4. Use negative examples to identify knowledge gaps
5. Auto-suggest knowledge base updates for frequently-failed queries

9.3 Memory Integration

- **Current:** Session-based chat history (lost on refresh)
- **Future:** Persistent conversation memory across sessions
- **Implementation:**

1. Use LangGraph's `PostgresSaver` checkpointer for state persistence
2. Store user profiles with interaction history
3. Use conversation summaries for long-term context
4. Implement a "memory" node that retrieves relevant past interactions

9.4 Deployment

- **Current:** Local Streamlit instance
- **Future:** Cloud-deployed production system
- **Implementation:**

1. Containerize with Docker
2. Deploy to cloud (AWS ECS, GCP Cloud Run, or Streamlit Cloud)
3. Add authentication (Streamlit's built-in auth or external OAuth)
4. Set up monitoring (LangSmith + cloud metrics)
5. Configure auto-scaling based on query volume

9.5 Additional Enhancements

- **Hybrid Search:** Combine vector search with BM25 keyword search for better retrieval
- **Re-ranking:** Add a cross-encoder re-ranker to improve retrieved document relevance
- **Streaming Responses:** Stream LLM output token-by-token for better UX
- **Multi-language:** Support queries in multiple languages
- **Analytics Dashboard:** Track query patterns, escalation rates, and response quality
- **Automated KB Updates:** Detect outdated information and suggest updates

---

End of Technical Documentation